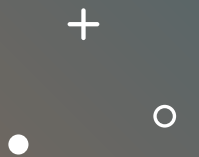
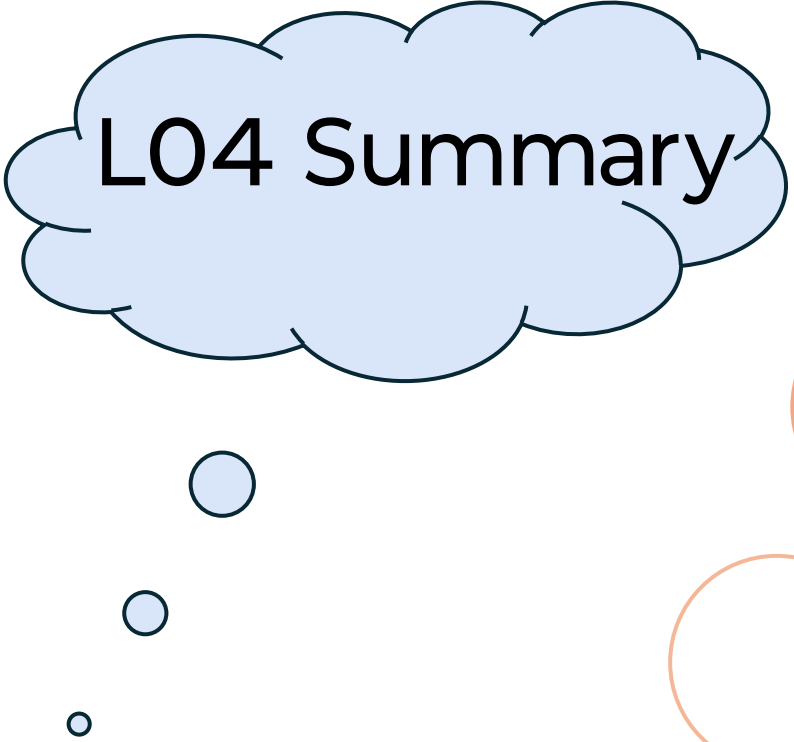


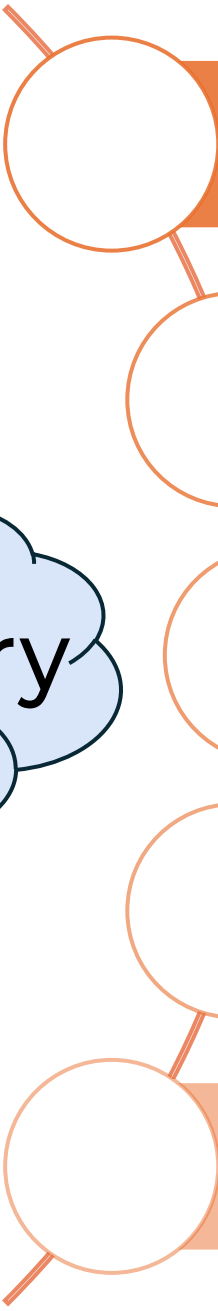


L04: Object Detection using YOLOV5





L04 Summary



Recap L03

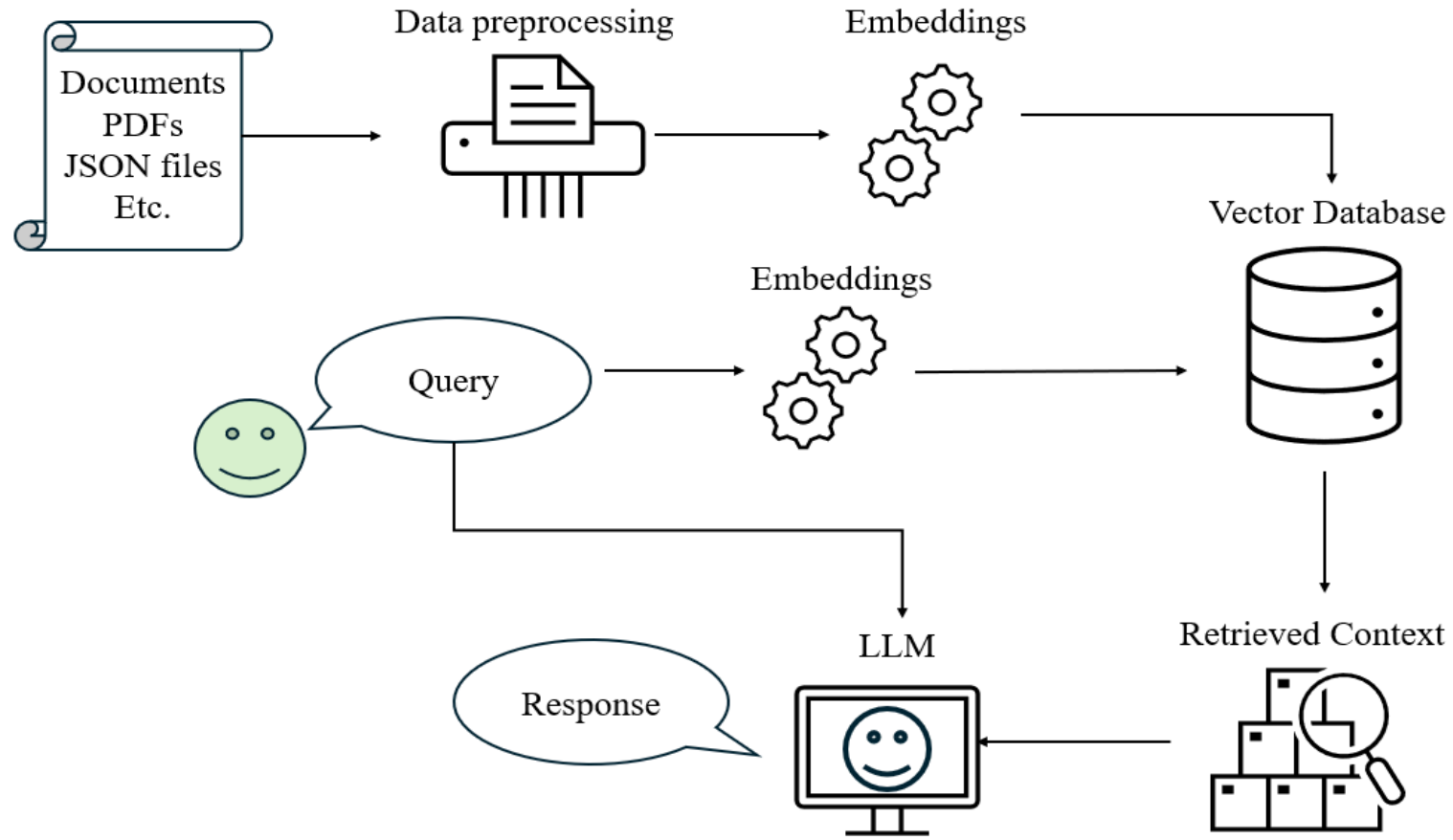
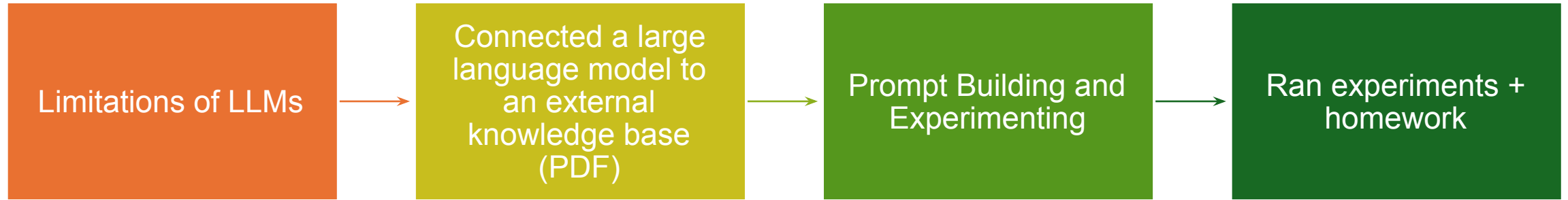
Introduction to Object Detection

YOLOv5

Steps Overview

Home Assignment

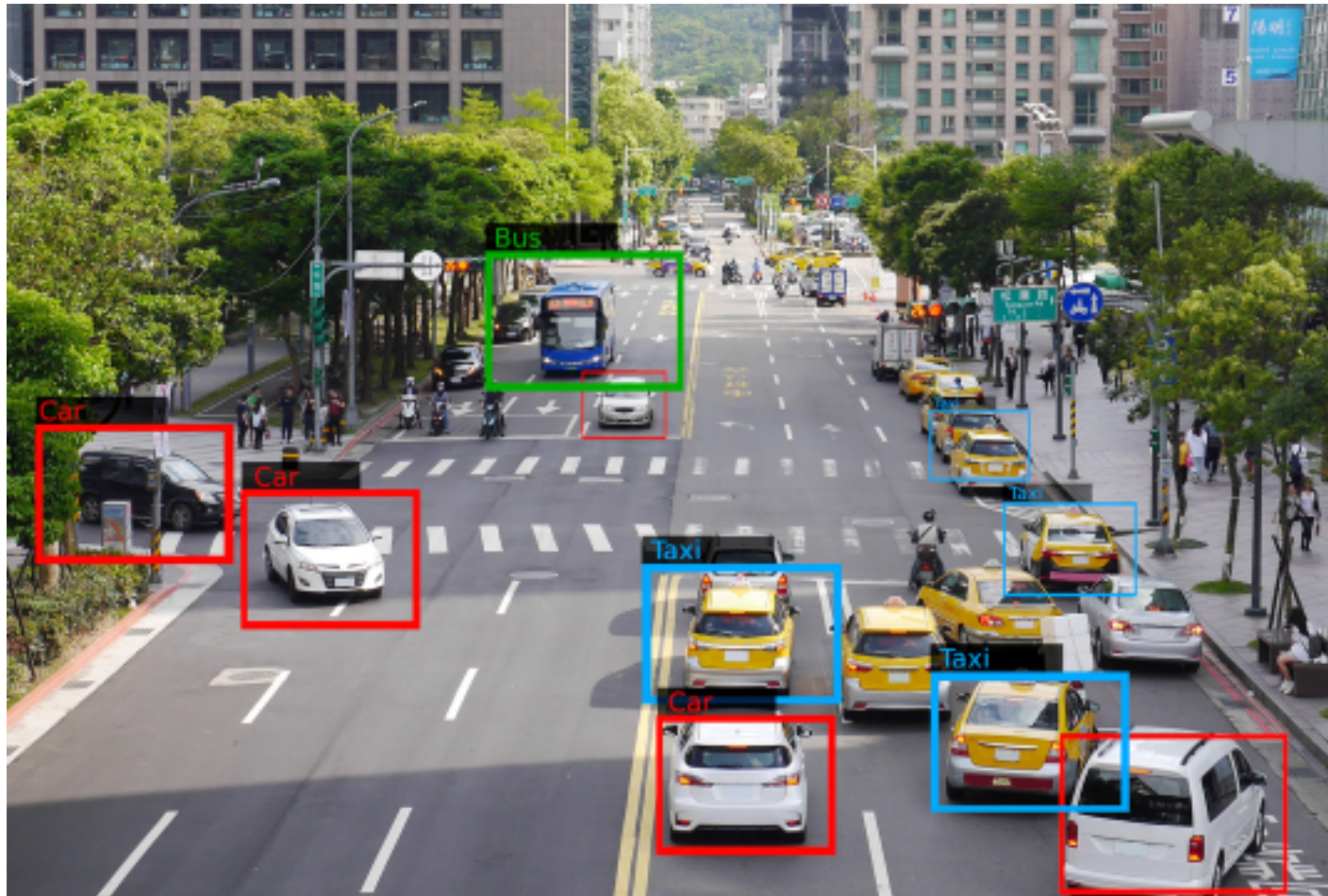
Recap L03



Introduction to Object Detection

Object Detection = Image Classification + Object Localization

- Image Classification: Recognizes and categorizes objects ("This image contains a cat")
- Object Localization: Goes a step further by identifying the precise location of objects



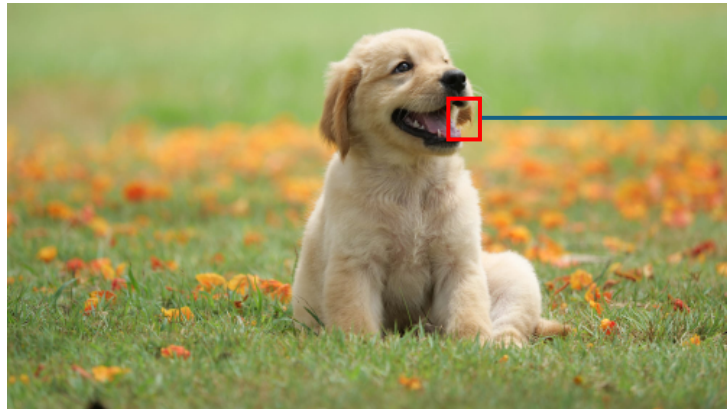
The Computer Vision Hierarchy

Computer Vision addresses three related, but distinct, problems when analyzing an image:

Task	Goal	Output	Analogy
Image Classification	Determine what is in the image.	A single class label (e.g., "Cat")	"Is there an animal in this photo? Yes, it's a cat."
Object Localization	Determine what is in the image and where it is.	A single class label + one bounding box	"Where is the main cat in the photo? Here it is."
Object Detection	Determine what and where all relevant objects are.	Multiple class labels + multiple bounding boxes	"Are there multiple animals, and where exactly are they? A cat here, and a dog there."

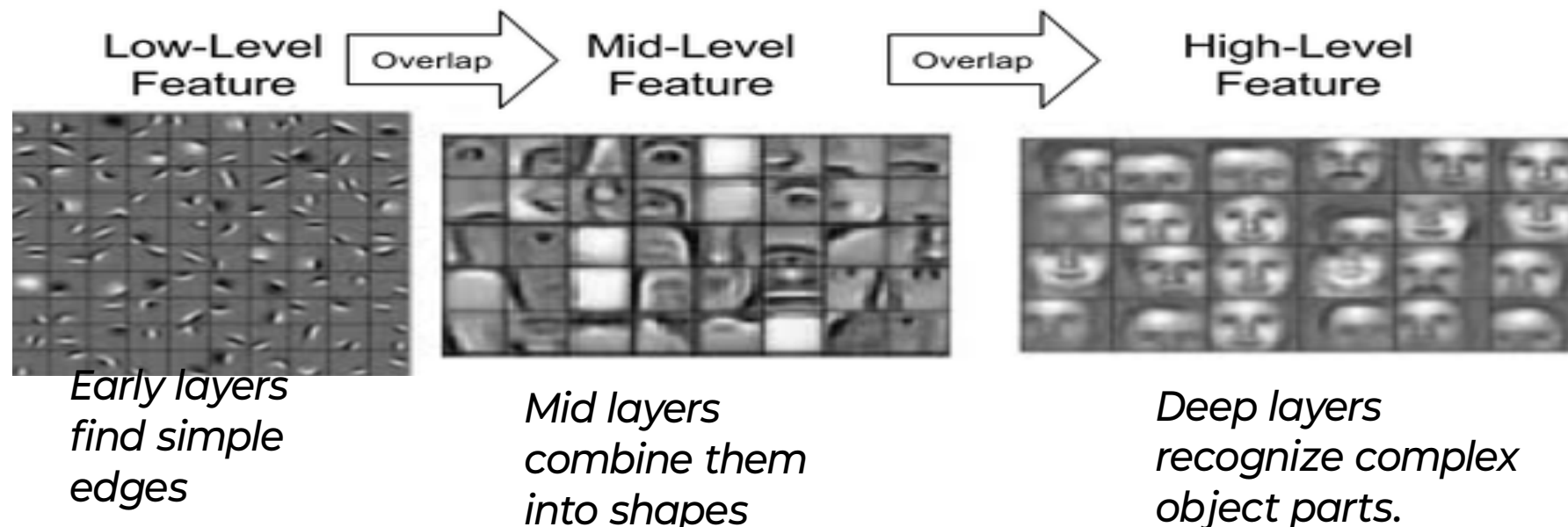
How a Model “Sees” an Image

Image = Matrix of Pixels (RGB values, 0-255)



Every image is made of pixels
Each pixel has 3 values (R,G,B) from 0 to 255

Before training, the pixel values are scaled (divided by 255) => range [0, 1]
Convolutional Neural Networks (CNNs)



From Two-Stage to One-Stage Detectors

The field of object detection saw a **major shift** from two-stage methods to fast, real-time single-shot methods.

Two-Stage Detectors (e.g., R-CNN, Faster R-CNN)

Process: The models perform detection in two separate, sequential steps.

1. Region Proposal: a separate algorithm (like Selective Search or Region Proposal Network) suggests potential bounding boxes where an object might be
2. Classification and Refinement: each proposed region is passed through the main CNN for classification and precise bounding box adjustment (regression)

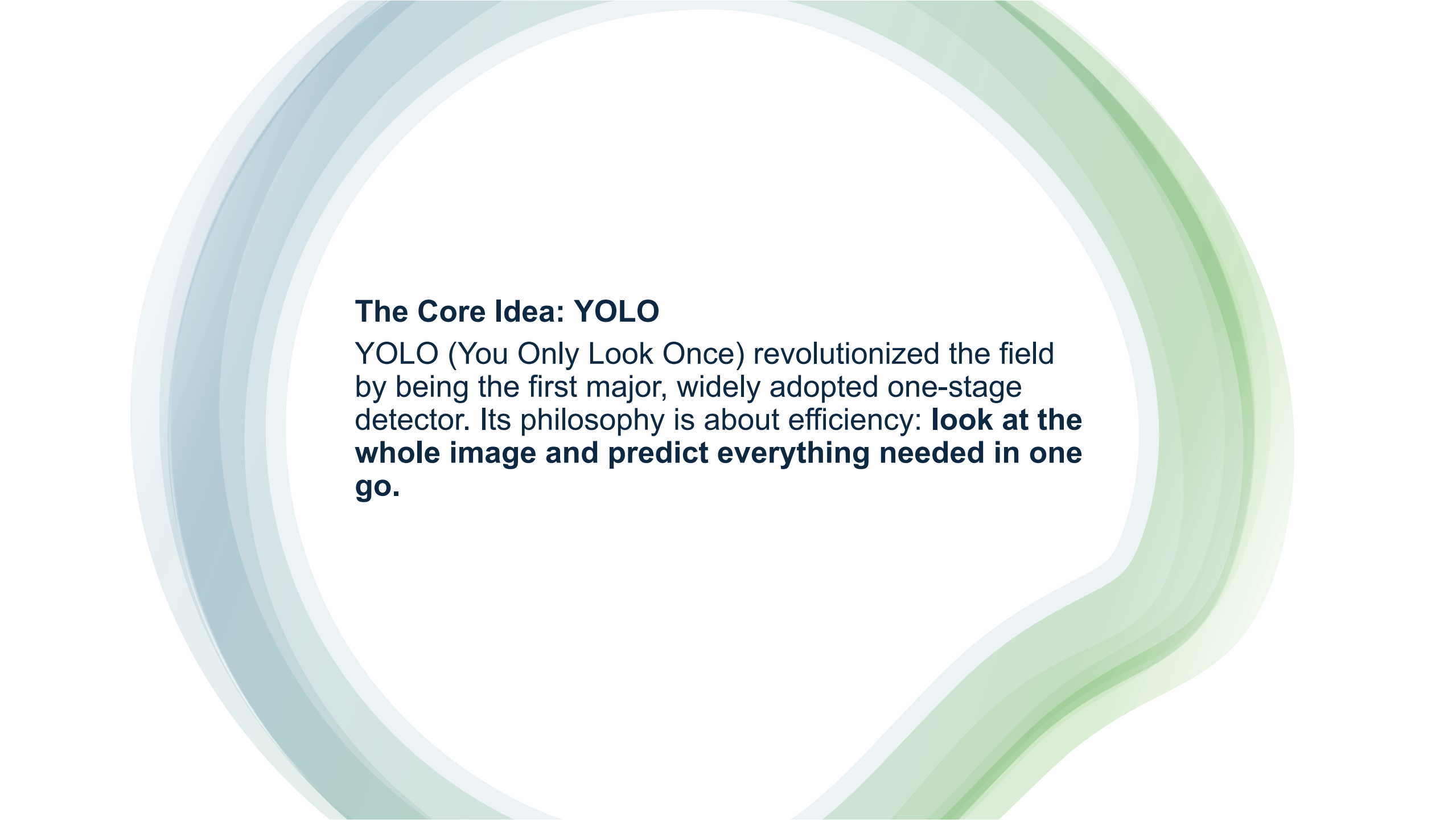
High **Accuracy**, but **Slow**

One-Stage (Single-Shot) Detectors (e.g., YOLO, SSD)

Process: The entire image is processed by a single neural network in one pass. It predicts all boxes and classes simultaneously.

It treats object detection as a regression problem

High Speed (Real-Time), with comparable accuracy



The Core Idea: YOLO

YOLO (You Only Look Once) revolutionized the field by being the first major, widely adopted one-stage detector. Its philosophy is about efficiency: **look at the whole image and predict everything needed in one go.**



Why YOLO?

YOLOv5 models are pre-trained on the **COCO dataset** (Common Objects in Context), which contains over **330,000 images** with **80 object categories** like *person, dog, car, and bottle*.

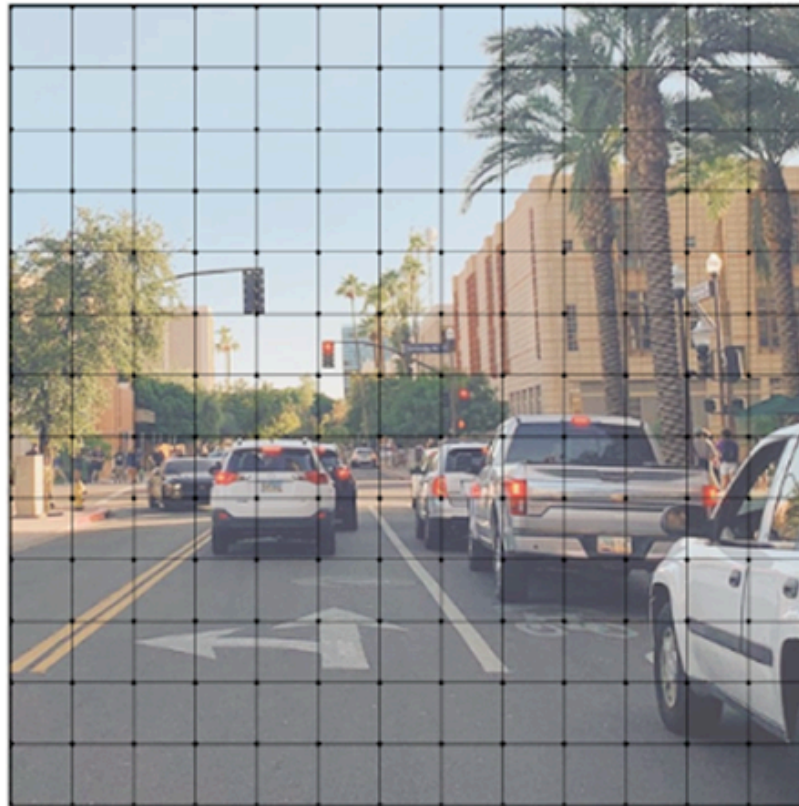
Why YOLO?

- ✓ Speed
- ✓ Detection accuracy
- ✓ Good generalization
- ✓ **Open-source**

The Core Mechanism: The Grid

The first step is dividing the complete (whole) frame into smaller boxes or grids

Each cell in the grid is responsible for localizing and predicting the class of the object that it contains.

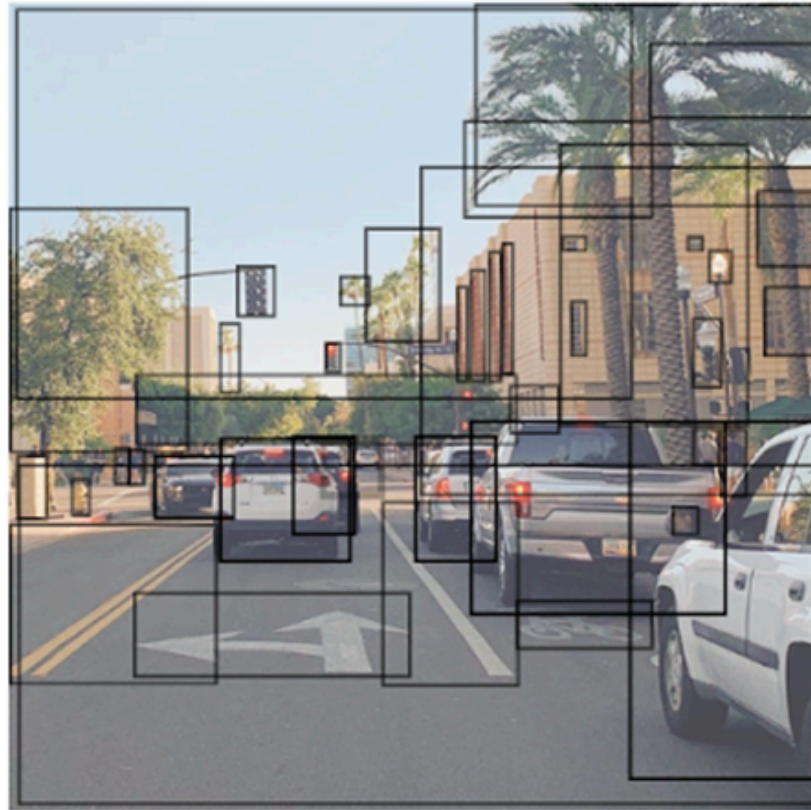


A grid cell is responsible for detecting an object **if, and only if, the CENTER POINT of that object falls inside that cell.**

The original image (A) into NxN grid cells of equal shape

Bounding box regression

- The next step is to determine the bounding boxes corresponding to rectangles, highlighting all the objects in the image.
- YOLO determines the attributes of these bounding boxes using a single regression module in the following format, where Y is the final vector representation for each bounding box.



$$Y = [pc, bx, by, bh, bw, c1, c2, \dots, cn]$$

pc = probability score of the grid containing an object.

bx, by are the x and y coordinates of the center of the bounding box

bh, bw = height and the width of the bounding box

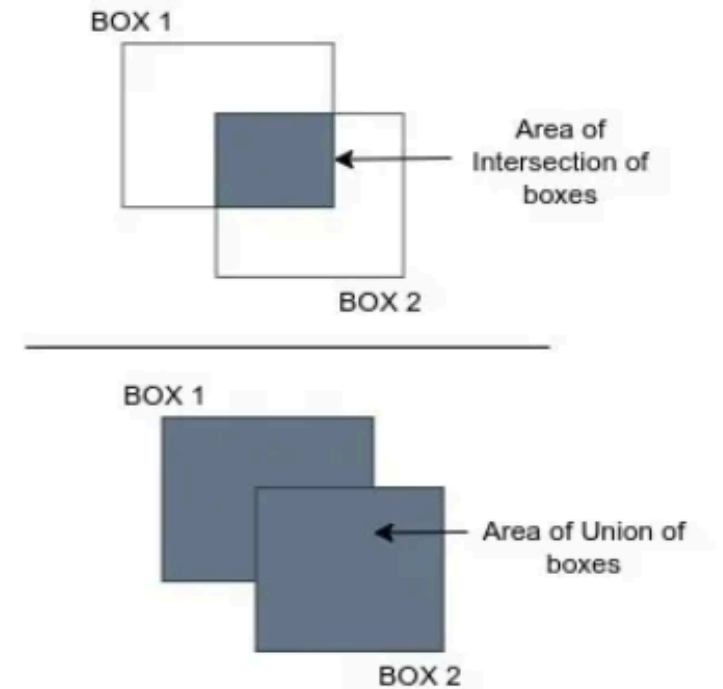
c1, c2...cn corresponding to the classes of objects

Intersection over Union

- Goal: Only keep the **relevant boxes**, discard the overlapping boxes to clean the mess.
- How? Using IOU
- The user defines its IOU selection threshold
 - IOU = 1 => perfect overlap
 - IOU = 0 => no overlap

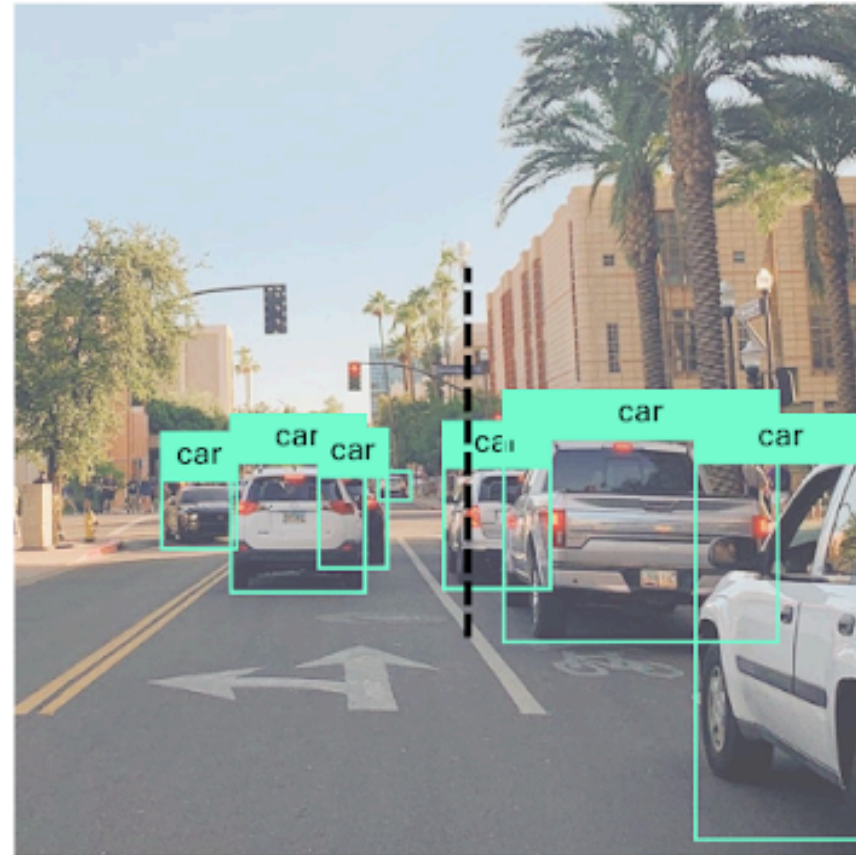
$$IOU = \frac{\text{Area of Intersection of two boxes}}{\text{Area of Union of two boxes}}$$

IOU =

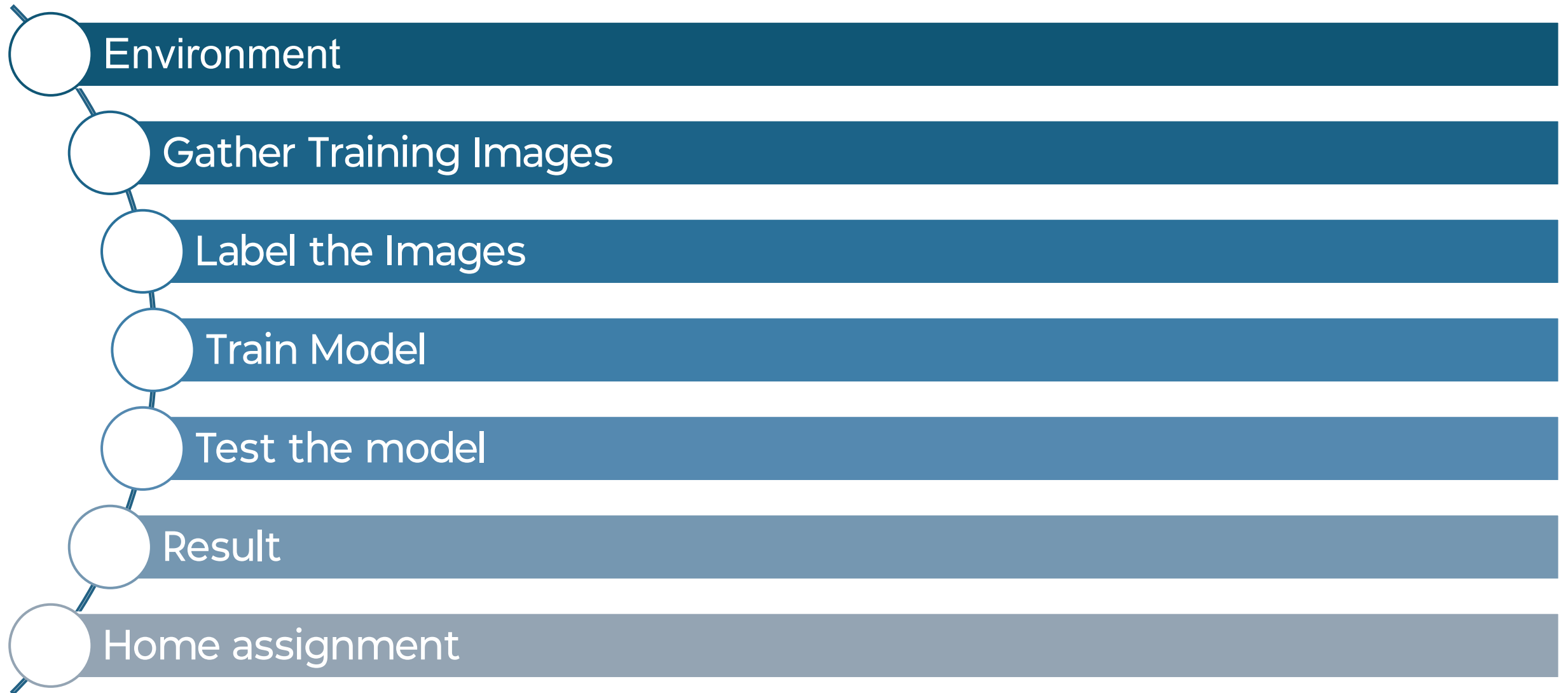


Non-Max Suppression (NMS)

- NMS (Non-Maximum Suppression) is the algorithm that cleans up duplicate predictions.
 1. It selects the boxes with the highest confidence score
 2. It uses then IOU to compare the best box to all other boxes
 3. Any box with a high IOU ($>$ threshold) with the best box is deleted (duplicate)



Detecting images using YOLOv5 - Overview of Steps



1. Setting up the Environment

- Open Google Colab

Make sure that GPU is selected!

- Clone YOLOv5 repository

```
!git clone https://github.com/ultralytics/yolov5
```

```
%cd yolov5
```

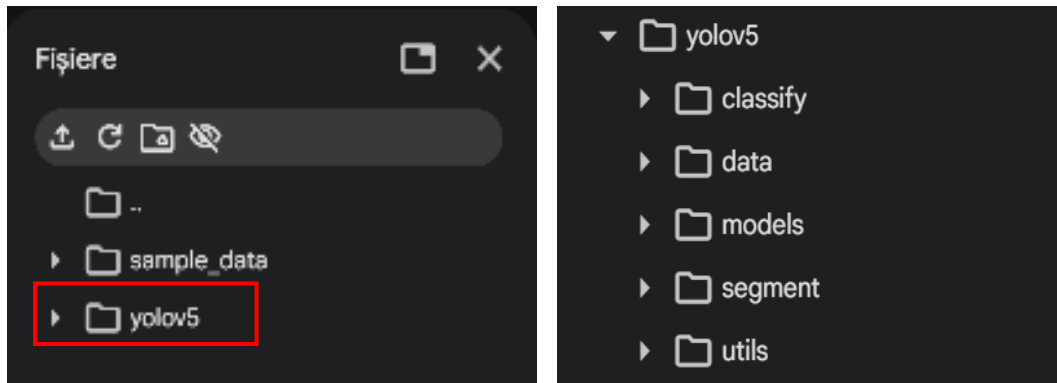
- Install dependencies

```
!pip install -r requirements.txt
```

- This includes:
 - o torch, torchvision: PyTorch framework for deep learning and computer vision tasks. YOLOv5 is built on PyTorch.
 - o Numpy: Numerical operations and matrix handling (used for images, bounding boxes,)
 - o opencv-python: Image and video resizing, augmentations and drawing bounding boxes.
 - o Matplotlib: Plotting training curves and visualizing detections and results.
 - o PyYAML: Parsing .yaml files (for example data.yaml, model configs).
 - o Pandas: Tabular data like annotations, metrics, training logs.
 - o scipy: Scientific computing, math operations

ultralytics/yolov5.git

It is the **official GitHub repository** for **YOLOv5**, maintained by **Ultralytics**, a company focused on computer vision.



	• train.py • detect.py • val.py • export.py
	• coco.yaml • hyp.scratch-low.yaml
	• train.py • Predict.py • val.py
	• yolov5s.yaml • yolov5m.yaml • yolov5l.yaml • Yolov5x.yaml
	• plots.py • metrics.py • loss.py

2. Data Preparation

A. Using an Existing Dataset – COCO

- o Test the model
- o Use the pre-trained weights which were already trained on the 80 classes

B. Custom Data

- Uses the model's existing knowledge
- Requires new images and labels to teach the model new classes.

Custom Data

What you need:

- A folder of **images** (JPEG, PNG, etc.)
- Each image should contain the objects you want to detect
- Should have random objects in the image along with the desired objects,
- Should have a variety of backgrounds and lighting conditions.

These images need to be labeled, then put into the YOLO format for the training.

Custom Data - The YOLO format

For every image (.jpg), there must be a corresponding text file (.txt) with a single line for each bounding box

`<class_id> <center_x> <center_y> <width> <height>`

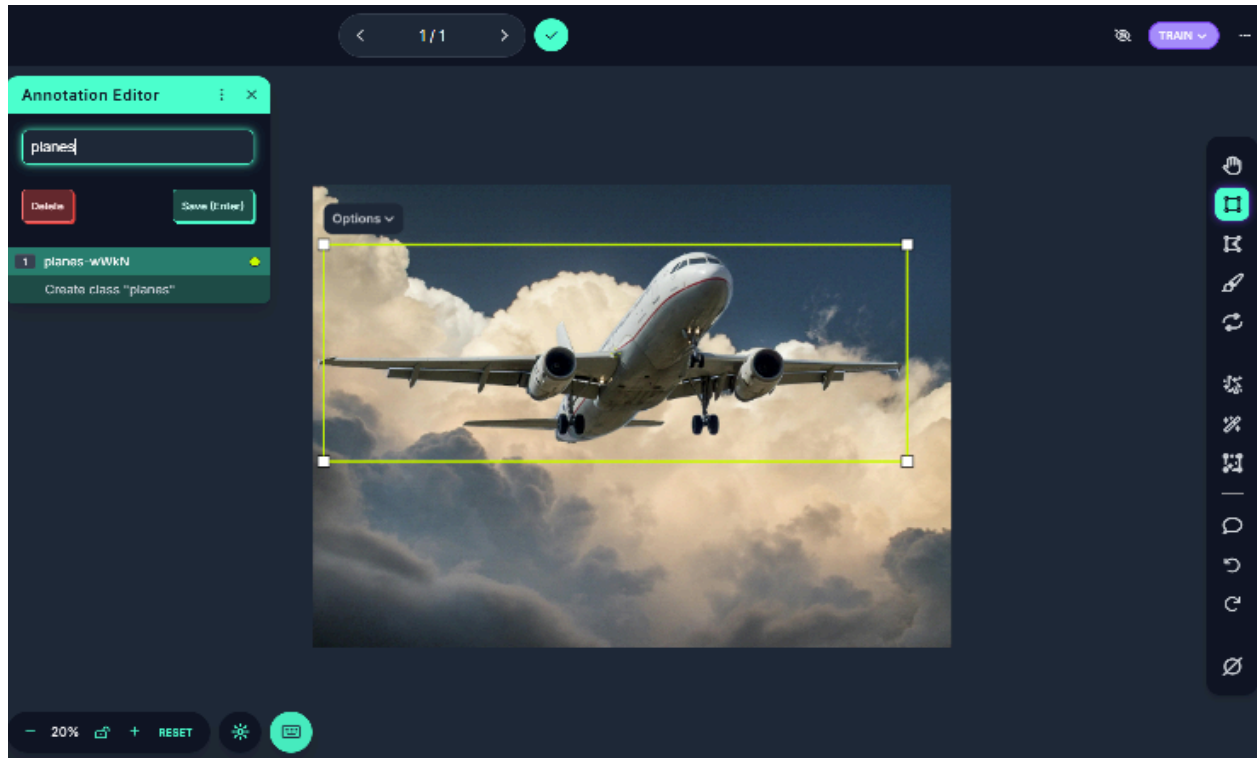


Custom Data

To label the images and create the bounding boxes, you can use:

- Roboflow (<https://app.roboflow.com/>)
- <https://www.makesense.ai/In>

After setting the bounding boxes and labeling, convert the dataset into the YOLOv5 format



Download

Image and Annotation Format

YOLO v5 PyTorch

YOLO v5 PyTorch

YOLO v7 PyTorch

Custom Data

Data must be then split into training and validation sets for learning and testing.

The folder of the images should have the following structure:

Usually, the training data represents 70%-80%
and the validation data is the rest.

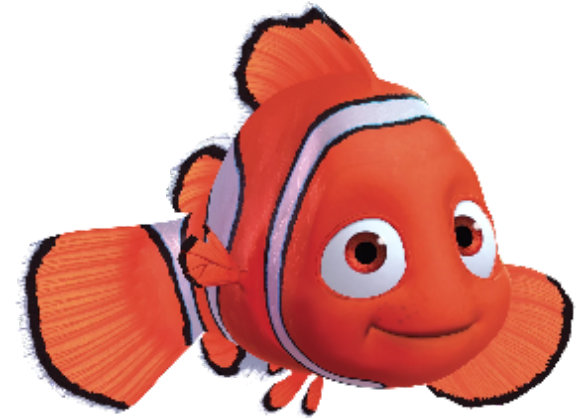
If you're using **Roboflow** - automatically splits the data.

```
dataset/  
├─ data.yaml  
├─ train/  
│   ├── images/  
│   │   ├── image1.jpg  
│   │   └── image2.jpg  
│   └── labels/  
│       ├── image1.txt  
│       └── image2.txt  
└─ val/  
    ├── images/  
    │   ├── image3.jpg  
    │   └── image4.jpg  
    └── labels/  
        ├── image3.txt  
        └── image4.txt
```

Tips from Roboflow to help you build a good model

- Label every relevant object in every image, even if it's partially visible
- Create tight bounding boxes, but don't cut off part of the object.
- Your training data should be representative of what your model will see when deployed
- Train your first model after ~70 images are labeled

Load Dataset – Let's play a game!
Finding Nemo 😊



This lab's task is to train the model to detect clownfish!

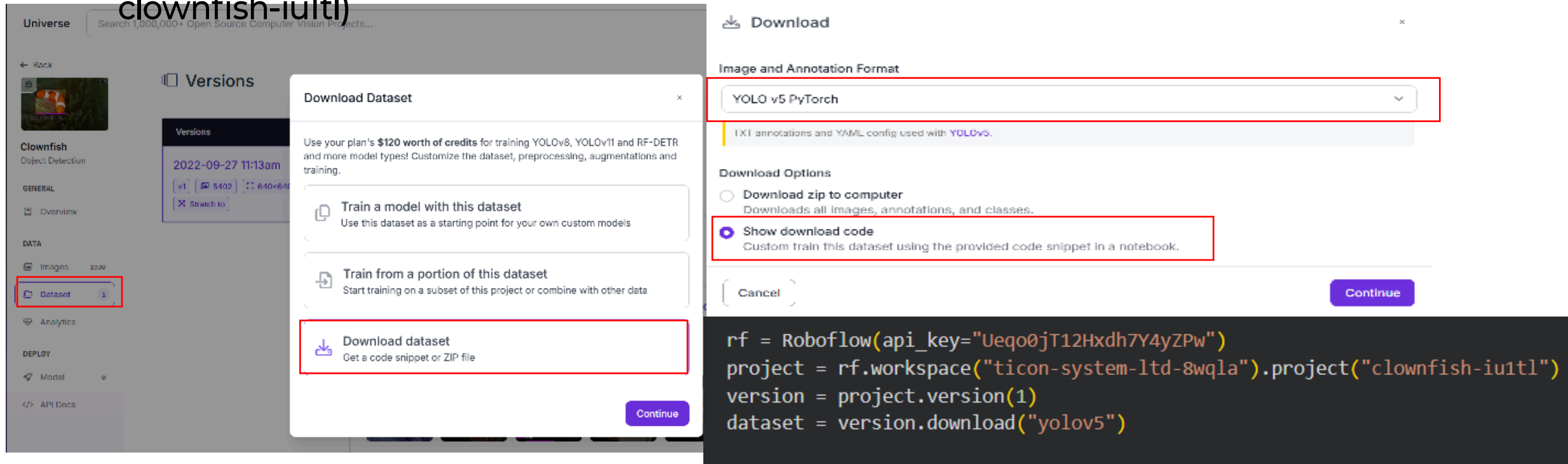
Load Dataset

Instead of manually collecting and labeling images, we will use **Roboflow** to export a dataset directly into Google Colab.

Roboflow is a platform that hosts datasets and automatically converts them into various formats (including YOLOv5).

1. Install roboflow `!pip install roboflow`

2. Load dataset from Roboflow (<https://universe.roboflow.com/ticon-system-ltd-8wqla/clownfish-iu1tl>)



The screenshot displays the Roboflow web interface. On the left, the 'Clownfish' dataset is selected under the 'Versions' tab. A 'Download Dataset' modal is open, showing three options: 'Train a model with this dataset', 'Train from a portion of this dataset', and 'Download dataset' (highlighted with a red box). To the right, the 'Download' panel is visible, showing the 'Image and Annotation Format' set to 'YOLO v5 PyTorch' (highlighted with a red box) and 'Download Options' where 'Show download code' is selected (also highlighted with a red box). Below the panels, a code snippet is shown for loading the dataset into a notebook.

```
rf = Roboflow(api_key="Ueqo0jT12Hxdh7Y4yZPw")
project = rf.workspace("ticon-system-ltd-8wqla").project("clownfish-iu1tl")
version = project.version(1)
dataset = version.download("yolov5")
```

Train Model

```
!python train.py --img 640 \  
                  --batch 8 \  
                  --epochs 5 \  
                  --data /content/yolov5/Clownfish-1/data.yaml \  
                  --weights yolov5s.pt \  
                  --name clownfish_yolov5s
```

Parameter	Description
!python train.py	Executes the training script => Transfer Learning
--img 640	The size to which all images are resized (640x640 pixels) This is a standard value, good practice for balance between speed and accuracy
--batch 8	The number of images processed simultaneously by the GPU before updating the model's weights Small value (Google Colab free plan)
--epochs 5(?)	The number of full cycles the model will run through the entire training dataset A very low number (5) to ensure the training finishes quickly
--data	location of images/labels and class names
--weights yolo5s.pt	Path to the initial weights file
--name	The name for the results (under runs/train/)

Test the model

The Goal: Generalization

The purpose of the final test is to verify **generalization**

1. Search on the internet for several pictures of clownfish
2. Load the pictures in Google Colab – you can put them in a folder “clownfish”
3. Use the picture as source when testing the model

```
!python detect.py \  
  --weights runs/train/clownfish_yolov5s2/weights/best.pt \  
  --img 640 \  
  --conf 0.6 \  
  --source /content/clownfish
```

Results

```
Validating runs/train/clownfish_yolov5s2/weights/best.pt...
```

```
Fusing layers...
```

```
Model summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
```

```
Class      Images  Instances    P          R    mAP50  mAP50-95: 100% 13/13 [00:05<00:00, 2.49it/s]
```

```
all        194      425    0.745    0.886    0.862    0.491
```

```
Results saved to runs/train/clownfish_yolov5s2
```

P (Precision)	0.745	How "trustworthy" the model's predictions are. Of all predictions made, how many of them were actually correct
R (Recall)	0.886	The ability of the model to find all the relevant cases within a dataset.
mAP50 (mean Average Precision)	0.862	Calculated with the IOU threshold fixed at 0.5
mAP50-95 (	0.491	It calculates 10 different mAP scores (one at IOU 0.5, one at 0.55, one at 0.60, ... all the way to 0.95) and then averages them all.

Result – Display the results

```
import glob
from IPython.display import Image, display

for image_path in glob.glob('runs/detect/exp16/*.jpg'):
    display(Image(filename=image_path))
```



Home assignment



Label your own images
and use your own dataset



Train the model



Send a report: bianca.
ebrasu@gmail.com